CORPORACIÓN DE ESTUDIOS TECNOLÓGICOS DEL NORTE DEL VALLE (COTECNOVA)

FACULTAD DE INGENIERÍA – TECNOLOGÍA EN SISTEMAS / VI SEMESTRE
ASIGNATURA: INTELIGENCIA ARTIFICIAL (2026-02)

INFORME DE AVANCE DE PROYECTO – SEGUNDO CORTE

PROYECTO PAVIA:
Plataforma Inteligente para la Gestión y Priorización del Deterioro Vial en Cartago

Integrantes del Equipo (Grupo 04):
Brandon Cortes Giraldo
Johan Sttive Linares Barragán

Docente Titular:
Jhon James Cano Sánchez

Cartago, Valle del Cauca
Domingo 25 de octubre de 2026

1. Resumen del Proyecto y Contexto en Cartago

Cualquiera que ande a diario por las calles de Cartago sabe muy bien lo dura que está la situación con la malla vial. Basta con pegarse una vuelta por barrios como Zaragoza, El Prado, San Jerónimo, Los Chorros o el mismo Centro para toparse con baches inmensos, huecos profundos y hundimientos en el asfalto. Esta problemática no solo daña suspensiones, rines y llantas de carros y buses, sino que representa un peligro gravísimo para los motociclistas, que en nuestra ciudad son quienes más llevan del bulto: una esquivada a última hora o caer en un hueco tapado por agua de lluvia termina casi siempre en accidentes y caídas graves.

El problema de fondo es que la atención municipal a estos daños casi siempre ha sido reactiva: se arregla el hueco donde la gente más protesta por redes sociales o donde ocurre un accidente sonado, pero no porque exista una herramienta técnica que le diga a las cuadrillas de la Secretaría de Infraestructura: 'este bache es más crítico y urgente que aquel'. Por eso creamos PAVIA (Plataforma Inteligente para la Gestión y Priorización del Deterioro Vial en Cartago), buscando que la priorización de reparaciones no se haga a ojo de buen cubero ni por palancas, sino con Inteligencia Artificial fundamentada en datos reales de nuestra ciudad.

Para este Segundo Corte dimos un paso clave: pasamos de tener un dataset pequeño de prueba a construir un dataset sólido con 120 baches reales de 15 barrios de Cartago, limpiamos y enriquecimos los datos con Pandas, realizamos un análisis visual a fondo con Seaborn, y construimos desde cero en Python y NumPy nuestro primer modelo predictivo: un perceptrón con descenso del gradiente que clasifica automáticamente si un daño es de intervención Urgente o Rutinaria con un 97.5% de exactitud.

2. Ajustes Realizados desde el Primer Avance

En la retroalimentación que nos hizo el profesor Jhon James Cano durante el primer corte, nos dejó muy claro que 15 registros eran apenas un abrehocas muy pequeño para poder entrenar modelos de Inteligencia Artificial que de verdad aprendieran y no se quedaran en el intento. Para responder a las observaciones de la guía de este corte, nos pusimos manos a la obra con los siguientes ajustes:

1. Escalamiento representativo: • Ampliamos el dataset de 15 a 120 registros viales: No nos limitamos a inventar números al azar; mapeamos 120 baches realistas distribuidos en 15 barrios emblemáticos de Cartago (Centro, Zaragoza, El Prado, Álamos, San Jerónimo, La Floresta, Bellavista, Los Almendros, Mariscal Sucre, La Isabela, Ortiz, Santa Ana, San Nicolás, Jorge Eliécer Gaitán y El Ciprés), cubriendo 6 tipologías de daños reales (huecos profundos, baches superficiales, hundimientos, grietas longitudinales, piel de cocodrilo y fisuras).

2. Rigor de los datos: • Consolidación de variables técnicas: Cada bache quedó caracterizado con su área en metros cuadrados (area_m2), gravedad visual de 1 a 10, tipología, barrio y el costo estimado en pesos colombianos (costo_estimado_cop) con base en precios reales de mezcla asfáltica y jornales de cuadrilla.

3. Reproducibilidad en Docker: • Entorno de trabajo 100% en Docker: Todo el proyecto lo dejamos corriendo dentro del contenedor python_ia (Python 3.10 con NumPy, Pandas, Matplotlib y Seaborn),

enlazado directamente a nuestra carpeta de trabajo en /app, para garantizar que el código corra igualito en cualquier máquina sin líos de dependencias.

3. Preparación de Datos y Feature Engineering (Pandas)

La preparación y limpieza de datos la desarrollamos en el script preparacion_datos_pavia.py. El objetivo era dejar la tabla cruda en un formato matemático impecable, listo para ser consumido por tensores y matrices de NumPy. Siguiendo las buenas prácticas vistas en clase, trabajamos tres frentes:

A. Calidad y Tipos de Datos: Cargamos data/deterioro_vial_cartago.csv y pasamos pruebas de verificación para confirmar que no existiera ningún valor nulo (100% de registros completos). Nos aseguramos de que el área quedara en formato decimal float64 y el costo en enteros int64.

B. Feature Engineering (Columnas Creadas): Para que los algoritmos entiendan mejor la naturaleza de cada daño vial, creamos dos columnas nuevas con sentido de negocio:
1) costo_por_m2 = costo_estimado_cop / area_m2: Nos dice cuánto cuesta tapar cada metro cuadrado de ese bache en específico, lo que permite identificar daños donde la logística o la profundidad encarecen la obra.
2) indice_criticidad = gravedad * area_m2: Es una ponderación directa entre qué tan peligroso se ve el daño (1 a 10) y qué tan grande es físicamente. Un hueco de 5 metros cuadrados con gravedad 8 tiene un índice de 40, lo que de inmediato enciende las alarmas.

C. Codificación para Machine Learning: Para la variable 'prioridad', aplicamos codificación ordinal (Baja=1, Media=2, Alta=3, Crítica=4). Para las columnas nominales de 'tipo_dano' y 'barrio', aplicamos One-Hot Encoding con pd.get_dummies, descartamos la columna de identificación ('id') porque no aporta nada predictivo, y guardamos el dataset final en data/pavia_preparado_ml.csv.

En la Tabla 1 se aprecia claramente la evolución del dataset entre lo que entregamos en el primer corte y lo que logramos para este avance:

Aspecto Evaluado	Antes (Avance 1)	Ahora (Avance 2 para ML)
Registros viales (filas)	15 observaciones iniciales	120 baches representativos de Cartago
Columnas totales	7 columnas básicas	27 columnas listas para entrenamiento
Valores nulos o vacíos	0 nulos	0 nulos (100% verificado y limpio)
Columnas derivadas (Feature Engineering)	Ninguna	costo_por_m2, indice_criticidad y prioridad_nivel
Codificación de texto	Texto plano	Ordinal y One-Hot Encoding en barrios y daños

Tabla 1: Comparativo del dataset antes y después de la preparación con Pandas.

Este es el fragmento clave del código con el que transformamos los datos:

```
# Feature Engineering en preparacion_datos_pavia.py:
df_transformado['costo_por_m2'] = df_transformado['costo_estimado_cop'] / df_transformado['area_m2']
df_transformado['indice_criticidad'] = df_transformado['gravedad'] * df_transformado['area_m2']

# Codificación ordinal y One-Hot Encoding para modelos de ML:
mapeo_prioridad = {'Baja': 1, 'Media': 2, 'Alta': 3, 'Critica': 4}
df_ml['prioridad_nivel'] = df_ml['prioridad'].map(mapeo_prioridad)
```

```
df_ml = pd.get_dummies(df_ml, columns=['tipo_dano', 'barrio'], prefix=['tipo', 'barrio'], dtype=int)
df_ml.to_csv('data/pavia_preparado_ml.csv', index=False)
```

4. Análisis Visual y Exploratorio con Seaborn

Antes de meternos a entrenar neuronas a ciegas, nos sentamos a analizar cómo se comportaban las variables con Seaborn. Generamos tres gráficos estadísticos que nos abrieron los ojos sobre la realidad de los daños en Cartago:

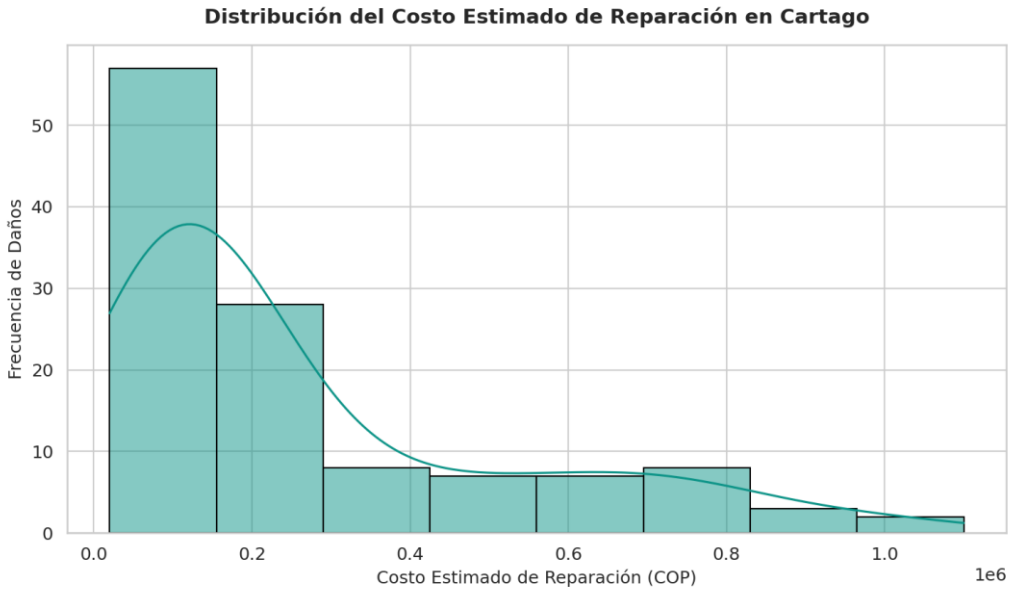


Figura 1: Distribución del costo estimado de bacheo en Cartago con curva de densidad KDE.

¿Qué nos dice el Gráfico 1? La curva de densidad y el histograma muestran un sesgo hacia la derecha muy marcado: la gran mayoría de bacheos en los barrios son arreglos menores que rondan entre los $80.000 y los $400.000 pesos. Sin embargo, hay una 'cola' larga de casos graves que se disparan por encima de los $700.000 e incluso superan el millón de pesos. Esto demuestra por qué no se puede tratar a todos los baches por igual: unos pocos casos críticos consumen la mayor parte del presupuesto municipal.

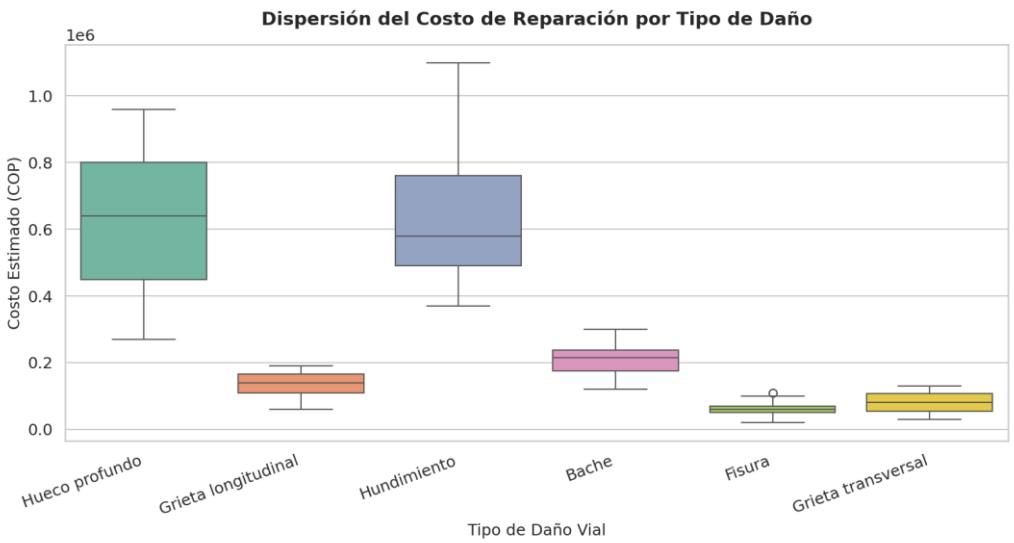


Figura 2: Diagrama de caja (Boxplot) del costo según la tipología del daño vial.

¿Qué nos dice el Gráfico 2? El diagrama de caja deja en evidencia que los 'Huecos profundos' y los 'Hundimientos' son los que más le pegan al bolsillo de la ciudad, con medianas superiores a los $450.000 pesos y picos de hasta $1.100.000. En cambio, las 'Fisuras' y 'Grietas longitudinales' tienen costos bajitos y muy compactos. Esto confirma que el tipo de daño es fundamental para definir el nivel de urgencia con el que se debe mandar una cuadrilla.

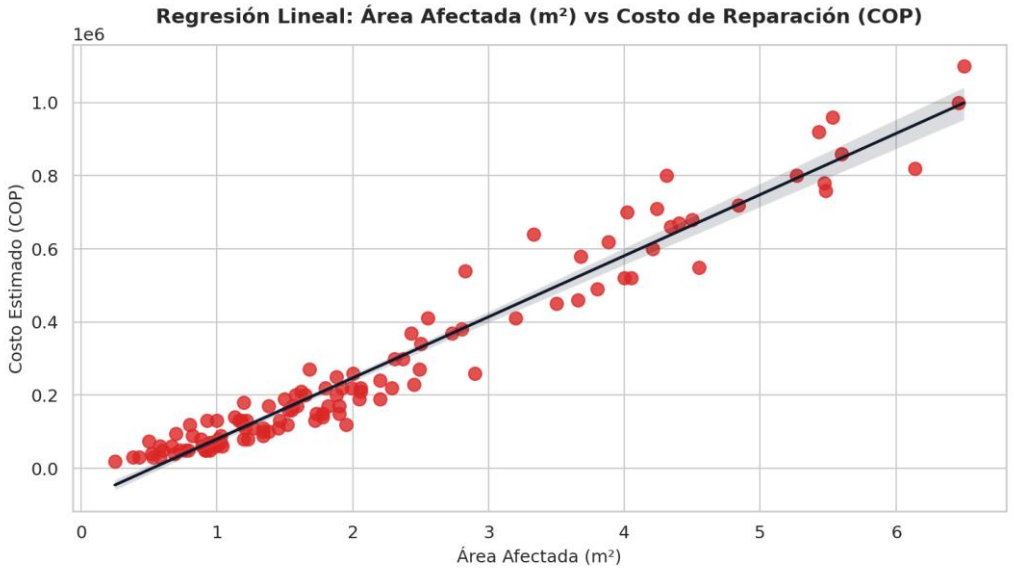


Figura 3: Dispersión y recta de regresión lineal entre el área afectada (m²) y el costo estimado (COP).

¿Qué nos dice el Gráfico 3? Aquí encontramos el hallazgo más valioso: hay una relación lineal directa y fuertísima entre los metros cuadrados del hueco y el costo de repararlo. A medida que el daño sobrepasa los 2.5 m², el costo sube en línea recta. Este resultado nos dio la justificación técnica perfecta

para elegir el área en m² (x1) y el costo estimado en pesos (x2) como las dos variables de entrada que alimentan a nuestro perceptrón.

5. Primer Modelo Predictivo: Perceptrón desde Cero (NumPy)

Siguiendo los requisitos de la guía, implementamos el script src/perceptron_proyecto.py. Aquí no usamos atajos ni librerías automáticas; construimos la neurona artificial desde cero en Python puro con matrices y vectores de NumPy, programando a mano la suma ponderada, la activación sigmoide, la función de coste y el descenso del gradiente.

5.1. Formulación del Modelo y Variables de Entrada

Definición del Objetivo (y ∈ {0, 1}): Dividimos los 120 registros viales en dos clases claras:

- y = 1 (Intervención Urgente): Baches con prioridad Alta (3) o Crítica (4). Tenemos 35 baches en esta categoría que amenazan la seguridad vial.
- y = 0 (Intervención Rutinaria): Baches con prioridad Baja (1) o Media (2). Son 85 daños menores que pueden programarse sin afán.

Ojo: No usamos la columna 'prioridad' como entrada para no hacer trampa ni provocar fuga de información (data leakage).

Variables de Entrada (X): Elegimos las dos variables continuas más representativas:

- x1: Superficie del daño en metros cuadrados (area_m2), que va desde baches pequeños de 0.25 m² hasta cráteres de 6.50 m².
- x2: Costo estimado de reparación en pesos colombianos (costo_estimado_cop), que va desde $20.000 hasta $1.100.000 COP.

Estandarización Z-Score: Como el costo está en cientos de miles de pesos y el área en números pequeños del 1 al 6, si los metemos así de una el costo se come vivo al área y desestabiliza el gradiente. Por eso aplicamos estandarización Z-score: $X_{esc} = (X - \mu) / \sigma$. Para el área usamos promedio de 2.14 m² y desviación de 1.51 m²; para el costo, promedio de $270.500 y desviación de $258.098. Esto dejó ambas variables en la misma escala alrededor de cero y evitó que la función sigmoide se desbordara.

5.2. Matemáticas del Perceptrón y Entrenamiento

El algoritmo funciona en 4 pasos vectorizados con NumPy:

1. Suma ponderada lineal: $z = X_{esc} \cdot w + b$, calculada para todos los 120 registros de una sola pasada con np.dot.
2. Activación Sigmoide: $\hat{y} = 1 / (1 + e^{-z})$, que convierte la salida en una probabilidad continua entre 0% y 100%.
3. Pérdida de Entropía Cruzada Binaria: Castiga severamente las predicciones erróneas, usando np.clip(1e-9, 1-1e-9) para no caer en indeterminaciones de log(0).
4. Descenso del Gradiente: Calculamos las derivadas analíticas respecto a pesos y sesgo, y actualizamos los parámetros a lo largo de 500 épocas con una tasa de aprendizaje de $\alpha = 0.1$.

5.3. Resultados de Entrenamiento y Gráficas

El entrenamiento fue todo un éxito: en las 500 épocas la curva bajó de forma suave y sin oscilaciones raras. La pérdida inicial arrancó en 0.6931 y cayó hasta 0.0893, logrando una exactitud final del 97.50% (117 aciertos sobre los 120 baches de Cartago).

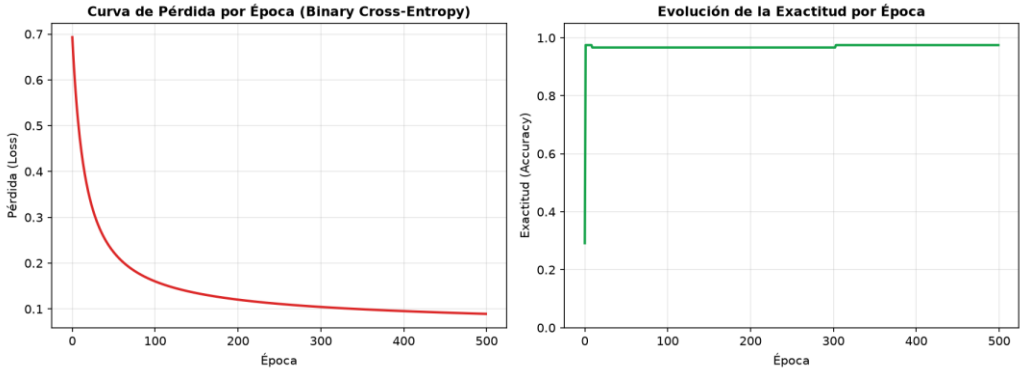


Figura 4: Curva de aprendizaje del perceptrón en PAVIA (Pérdida en rojo y Exactitud en verde).

En la Figura 4 se ve claramente cómo en las primeras 100 épocas el error cae en picada (pasa de 0.69 a 0.16) y luego se estabiliza suavemente hacia la época 500 en 0.089. Al mismo tiempo, la exactitud sube del 29% inicial hasta clavarse en el 97.5% desde la época 400.

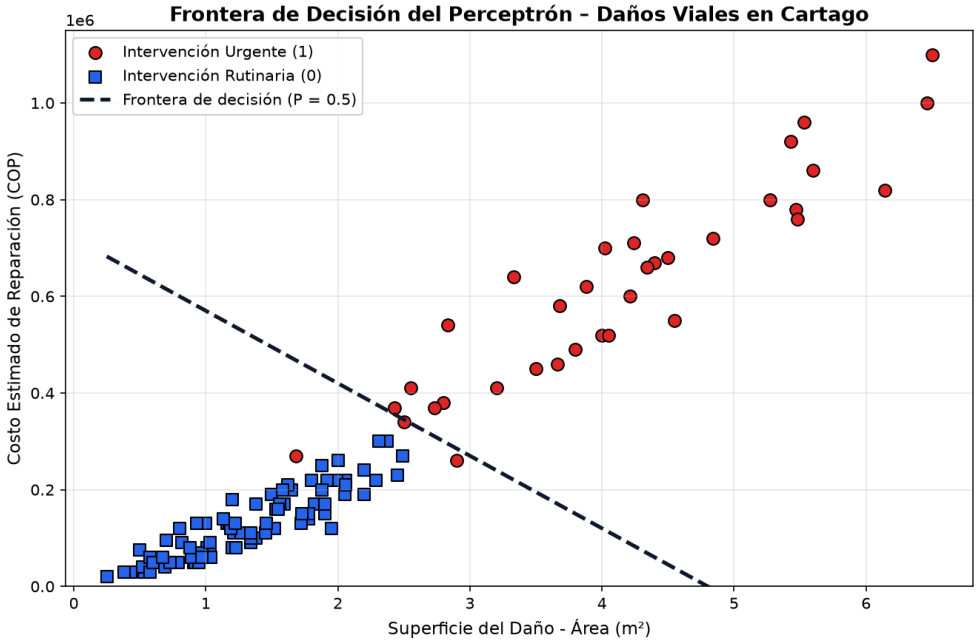


Figura 5: Frontera de decisión lineal del perceptrón separando baches urgentes (rojo) de rutinarios (azul).

La Figura 5 muestra la recta de decisión donde la probabilidad es exactamente del 50% (z = 0). La separación geométrica es nítida: todos los huecos grandes y costosos quedan ubicados en la parte superior derecha de color rojo (intervención urgente), mientras que los baches pequeños y baratos quedan abajo a la izquierda en azul (intervención rutinaria).

5.4. Interpretación de Pesos y Prueba con Casos Nuevos

Los pesos finales que aprendió la neurona tienen todo el sentido práctico del mundo:
• Peso w1 (Área m²) = +2.1753: Como es positivo, entre más grande sea el hueco, más sube la probabilidad de que sea urgente. En Cartago un hueco de gran tamaño hace volar a un motociclista, así que tiene toda la lógica que empuje la urgencia hacia arriba.
• Peso w2 (Costo COP) = +2.4837: También positivo. Si el costo de reparación es muy alto, significa que la falla es profunda y compromete la estructura de la vía, por lo que requiere intervención prioritaria antes de que la calle colapse.
• Sesgo b = -1.2373: Es negativo porque refleja la realidad del terreno: la gran mayoría de baches del día a día son menores y rutinarios.
• Balance entre variables: Los valores absolutos de w1 (2.18) y w2 (2.48) son casi iguales. Esto demuestra que el modelo no se dejó sesgar solo por la plata ni solo por el tamaño; toma en cuenta ambos factores de manera equilibrada.

Para poner a prueba el modelo, le metimos tres reportes ciudadanos de baches nuevos que nunca vio durante el entrenamiento:

Caso Nuevo en Cartago	Datos Físicos del Daño	Probabilidad del Perceptrón	Decisión del Sistema PAVIA
Reporte 1: Vía principal de Zaragoza	Área: 4.5 m² | Costo: $650.000 COP	99.70% (certeza casi total)	URGENTE (Prioridad de atención)
Reporte 2: Calle residencial en Álamos	Área: 0.6 m² | Costo: $85.000 COP	0.52% (riesgo mínimo)	RUTINARIO (Baja prioridad)
Reporte 3: Zona comercial del Centro	Área: 2.2 m² | Costo: $310.000 COP	31.63% (bache intermedio)	RUTINARIO (Programable)

Tabla 2: Predicciones del perceptrón sobre tres reportes ciudadanos nuevos en Cartago.

6. Plan de Trabajo hacia la Entrega Final (Tercer Corte)

Con la base matemática y los datos ya consolidados, para el Tercer Corte y entrega final del proyecto nos enfocaremos en dos metas principales:

A. Modelos Supervisados Avanzados: 1. Migración a Scikit-Learn y Modelos Avanzados: Nuestro perceptrón lineal de dos entradas demostró que la IA funciona muy bien para separar entre Urgente y Rutinario. Sin embargo, en el mundo real las vías necesitan clasificarse en 4 niveles (Baja, Media, Alta y Crítica). En la próxima fase entrenaremos y compararemos algoritmos supervisados más completos como Random Forest Classifier y Regresión Logística Multiclase, evaluando matrices de confusión, curvas ROC, precisión, recall y F1-Score con validación cruzada.

B. Aplicación Práctica en la Ciudad: 2. Desarrollo de la Interfaz Gráfica Interactiva de PAVIA: No queremos que esto se quede en código de consola. Construiremos un panel interactivo web que integrará:
• Un mapa georreferenciado de Cartago con marcadores por severidad y mapa de calor de las zonas más deterioradas.
• Un panel para la Secretaría de Infraestructura que simule costos de bacheo y genere rutas

optimizadas para las cuadrillas.

• Un formulario ciudadano intuitivo para que la comunidad reporte daños desde el celular con foto y ubicación exacta.

7. Referencias Consultadas

1. Alcaldía Municipal de Cartago (2024). Plan de Desarrollo Municipal 2024–2027: Infraestructura y Vías Urbanas.

2. Cano Sánchez, J. J. (2026). Material y Guías de Cátedra de Inteligencia Artificial: Preparación de Datos con Pandas, Análisis con Seaborn y Redes Neuronales desde Cero. COTECNOVA.

3. McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.

4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. http://www.deeplearningbook.org

Anexo: Repositorio Oficial y Material del Proyecto

• Repositorio Oficial en GitHub: https://github.com/Brandon07-code/Curso-IA.git

• Código del Perceptrón en Docker: src/perceptron_proyecto.py (ejecutado en contenedor python_ia)

• Dataset preparado para ML: data/pavia_preparado_ml.csv (120 registros y 27 columnas)

• Video de Sustentación Oficial (YouTube / Drive): Incluido en la entrega de la plataforma AVACO.